Final

May 2, 2021

[1]:
```
# written in Julia 1.6.0 by OIHernandez as Final backup for
# MATH F615 Numerical Analysis of Diff Eqs by DAMaxwell
# https://github.com/mathemonads/HW_615/blob/main/Final.ipynb

# import Pkg; Pkg.add("")
ENV["PYTHON"] = "python3"
```

[1]:
```
"python3"
```

[2]:
```
using BandedMatrices
using DataFrames
using Distances
using GLM
using LaTeXStrings
using LinearAlgebra
using NLsolve
using Roots
using ShiftedArrays
# set plotting environment
using PyCall
using Plots; pyplot()

dim  = (800,480)
rd(x)= round(x, digits=3)

# previous
function newton(f, Df, x0; tol=1e-9, nmax=1e9)
    f0 = maximum(abs.(f(x0)))
    xk = x0
    d = Df(xk)
    z = f(xk)
    xp = xk
    fk = 1
    k = 0
    while (k<nmax && abs(fk/(f0-fk)) > tol)
        k += 1
        p = -d\z
        xp = xk + p
```

```
        # terminate when residual norm has been reduced by the tolerance␣
↪specified
        fk = maximum(abs.(f(xp)))
        if abs(fk/(f0-fk)) < tol
            return (k, xp, fk)
        end
        xk = xp
        d = Df(xk)
        z = f(xk)
    end
    return (k, xp, fk)
end

function regress_convergence(h, Z)
    df = DataFrame(h=h, Z=Z)
    fm = @formula(log(Z) ~ 1 + log(h)) # log(Z) = log(C)*1 + p*log(h)
    lr = lm(fm, df) # fit a straight line to the data
    lC, p = GLM.coef(lr) # retreive constant coefficients log(C) and p
    C = exp(lC)
    return p, C
end
```

[2]: regress_convergence (generic function with 1 method)

1 Newton's method

Consider the problem

$$u_{xx} + \gamma u^4 = f(x)$$

on the interval $0 \leq x \leq 1$ with $u(0) = u(1) = 0$ for some constant γ.

1.1 Example

If $u(x) = sin(3\pi x)$, then the value of f is given by:

$$\begin{aligned} f(x) &= \frac{d}{dx}\left(3\pi\cos(3\pi x)\right) + \gamma\left(\sin(3\pi x)\right)^4 \\ &= -9\pi^2\sin(3\pi x) + \gamma\sin^4(3\pi x) \end{aligned} \tag{1}$$

1.2 Code

We write a code to solve this problem.

1.2.1 Approximation

We use centered differences as in Section 2.2 of our text to approximate the second derivative:

$$\frac{u(x_{i+1}) - 2u(x_i) + u(x_{i-1})}{h^2} + \gamma u(x_i)^4 \approx f(x_i) + O(h^2)$$

$$\frac{1}{h^2}\begin{pmatrix}1 & -2 & 1\end{pmatrix}\begin{pmatrix}u_{i-1}\\ u_i\\ u_{i+1}\end{pmatrix} + \gamma u_i^4 = f_i.$$

We use this to derive an algebraic system to solve for a vector $\vec{u}$ of unknowns u_i that approximate $u(x_i)$:

$$F(u) := A\vec{u} + \gamma\vec{u}^4 - \vec{f}(\vec{u}) = 0,$$

where $\left(\vec{f}(\vec{u})\right)_i = f(u_i)$, and A is the tridiagonal matrix with $\frac{-2}{h^2}$ on the diagonal and $\frac{1}{h^2}$ above and below the diagonal.

1.2.2 Implementation

We implement a numerical method to solve the system by applying Newton's method to approximate the solution of the algebraic system, with a user-supplied function f, constant γ, and number M of space steps. Newton's method stops when the residual norm $||F(u)||$ has been reduced by 10^{-9} of its original value.Newton's method stops when the residual norm $||F(u)||$ has been reduced multiplicatively by 10^{-9} of its original value.

[3]:
```
function genA(N, h)
    l = 1.0/h^2
    A = BandedMatrix((-1=>repeat([l],outer=[N-1]), 0=> repeat([-2*l],outer=[N]),␣
 ↪1=> repeat([l],outer=[N-1])), (N,N))
    return A
end

function solve(f, gamma, M)
    t0, u0, T = 0.0, 0.0, 1.0
    h = (T-t0)/M
    t = [t0:h:T;]
    u = zeros(length(t))
    u[1] = u0
    u[length(t)] = u0
    u_curr = u[2:length(t)-1]
    A = genA(length(u_curr), h)
    F(uk) = A*uk + gamma .* uk .^4 - f.(t[2:length(t)-1])
    Df(uk) = A + BandedMatrix(0 => [4*gamma*uj^3 for uj in uk])
    n, u[2:length(t)-1], fk = newton(F, Df, u_curr; tol=1e-9)
    return n, t, u, fk
end

u1_sol(x) = sin(3*pi*x)
```

```
function f1(gamma)
    f(x) = gamma*sin(3*pi*x)^4 -9*sin(3*pi*x)*pi^2
    return f
end
L"f(x) = \gamma\sin^4(3\pi x) - 9\sin(3\pi x)\pi^2"
```

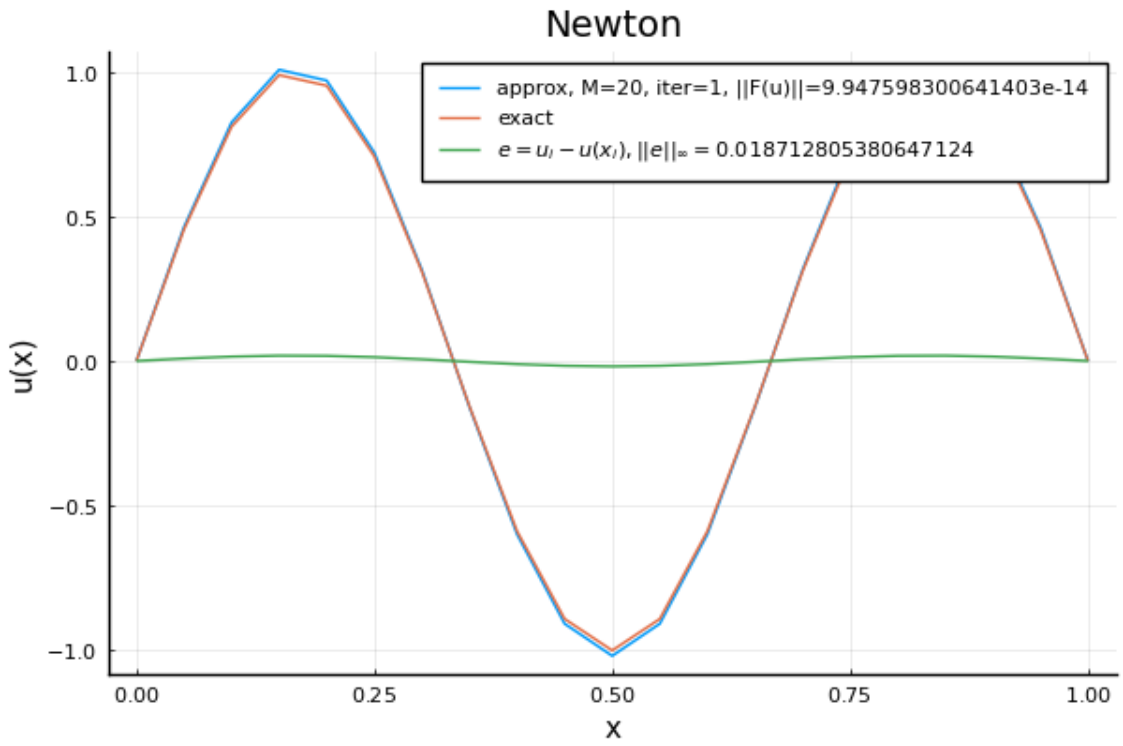
[3]: $f(x) = \gamma \sin^4(3\pi x) - 9\sin(3\pi x)\pi^2$

[4]:
```
function test_newton(gamma, M)
    n, t, u, fn = solve(f1(gamma), gamma, M)
    u_exact = u1_sol.(t)
    e = u-u_exact
    return n, t, u, fn, u_exact, e, maximum(abs.(e))
end

M = 20 # 100 is unreasonably good
gamma = 0.0
n1, t1, u1, f1n, u1_exact, e1, e1_inf = test_newton(gamma, M)


plot([t1 t1 t1], [u1 u1_exact e1], label=["approx, M=$M, iter=$n1,␣
 ↪||F(u)||=$f1n" "exact" L"e=u_i-u(x_i), ||e||_\infty=%$e1_inf"],␣
 ↪title="Newton", xlabel="x", ylabel="u(x)")
```

[4]:



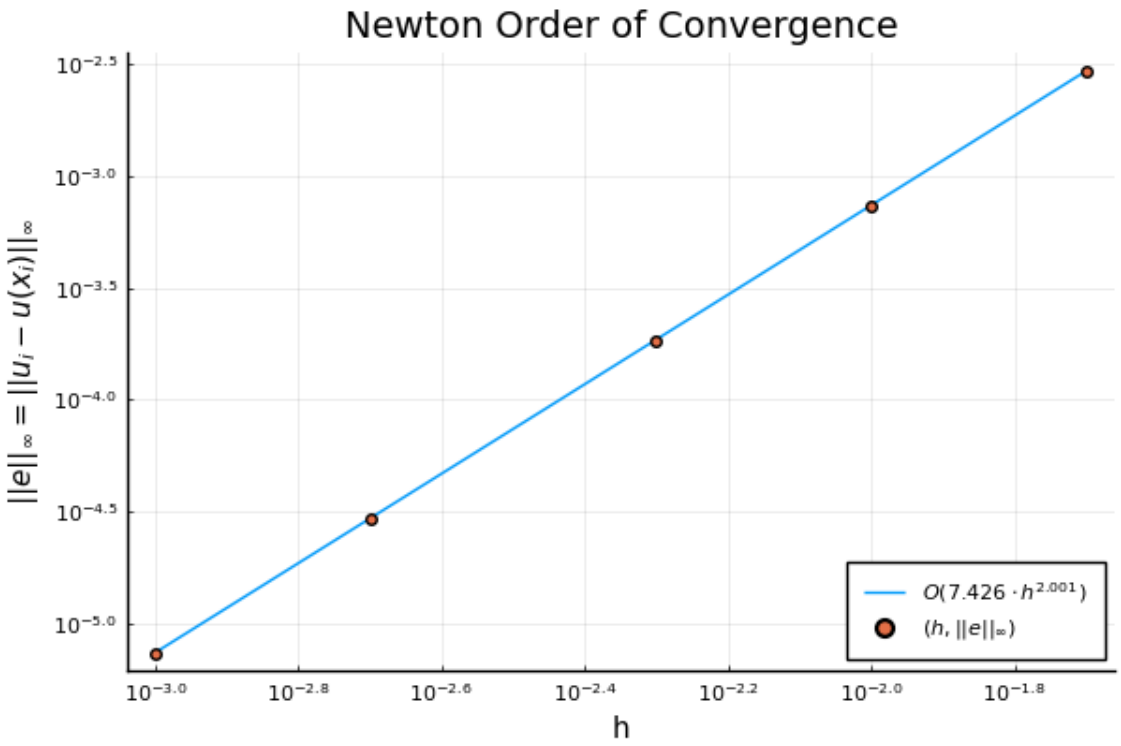
[5]:
```
function get_conv(Ms, gamma)
    t0, T = 0.0, 1.0
    H = zeros(length(Ms))
    Z = zeros(length(Ms))
    for i in 1:length(Ms)
        M = Int(round(Ms[i]))
        H[i] = (T-t0)/M
        n, t, u, fn, u_exact, e, e_inf = test_newton(gamma, M)
        Z[i] = e_inf
    end
    p, C = regress_convergence(H, Z)
    return H, Z, p, C
end

M1 = [50,100,200,500,1000]
H1, Z1, p1, C1 = get_conv(M1, gamma)
```

[5]:
```
([0.02, 0.01, 0.005, 0.002, 0.001], [0.0029661488369079336,
0.0007405492016876458, 0.0001850756315759927 2, 2.9609339089464726e-5,
7.402235903786192e-6], 2.000510813714256, 7.426099214037519)
```

[6]:
```
O1(h) = C1*h^p1
plot(O1, minimum(H1), maximum(H1), label=L"O(%$(rd(C1))\cdot h^{%$(rd(p1))})",␣
 ↪xaxis=:log, yaxis=:log, xlabel="h",␣
 ↪ylabel=L"||e||_\infty=||u_i-u(x_i)||_\infty")
scatter!(H1, Z1, legend=:bottomright, label=L"(h, ||e||_\infty)", title="Newton␣
 ↪Order of Convergence")
```

[6]:



1.2.3 Order of Convergence

With the verification case from part (a) and $\gamma = 0$, our code exhibits a roughly quadratic order of convergence:

$$O(7.426099214037519 \cdot h^{2.000510813714256}) \approx O(h^2).$$

2 Klein-Gordon

Consider the Klein-Gordon equation

$$c^2 u_{xx} = u_{tt} + bu$$

for positive constants $b, c > 0$ where the initial and boundary conditions are given by $u(x,0) = f(x)$, $u_t(x,0) = g(x) = 0$, and $u(0,t) = u(l,t) = 0$.

2.1 Explicit Finite Difference Method

Applying the centered-difference approximation for both second derivatives

$$c^2 \frac{u(x_{i+1},t_j) - 2u(x_i,t_j) + u(x_{i-1},t_j)}{h^2} + O(h^2) \approx \frac{u(x_i,t_{j+1}) - 2u(x_i,t_j) + u(x_i,t_{j-1})}{k^2} + O(k^2) + bu(x_i,t_j)$$

yields the following explicit finite difference method

$$u_{i,j+1} = 2u_{i,j} - u_{i,j-1} + \frac{c^2k^2}{h^2}\left(u_{i+1,j} - 2u_{i,j} + u_{i-1,j}\right) - bk^2u_{i,j}, \tag{2}$$

with $u_{i,0} = f_i = f(x_i)$, $u_{0,j} = u_{N,j} = 0$, and $\frac{u_{i,1}-u_{i,-1}}{2k} = g_i = 0$ for all $0 \leq i \leq N$ and $1 < j \leq M$. This yields $u_{i,-1} = u_{i,1}$ and thus

$$u_{i,-1} = u_{i,1} = = \frac{1}{2}\left[2f_i - + \frac{c^2k^2}{h^2}\left(f_{i+1} - 2f_i + f_{i-1}\right) - bk^2f_i\right].$$

This method is quadratic in space and time. Since it has the same analytic and numerical domains of dependence as the wave equation, this method satisfies the CFL condition.

2.2 Stability

Supposing $u_{i,j} = w^j e^{rx_iI} = w_j e^{rhiI}$, we obtain

$$\begin{aligned} w^2 - 2\left(1 - 2\lambda\sin^2(rh/2) - k^2/2\right)w + 1 &= 0 \\ -1 \leq 1 - 2\lambda\sin^2(rh/2) - k^2/2 &\leq 1 \\ \frac{k^2}{h^2} &\leq \frac{4}{4+h^2} \end{aligned}$$

2.3 Dispersivity and dissipativity

Suppose $u(x,t) = e^{I(kx-wt)}$. Then, the positivity of b, c, k imply that

$$\begin{aligned} 0 &= (Ik)^2c^2e^{I(kx-wt)} - (Iw)^2e^{I(kx-wt)} - be^{I(kx-wt)} \\ 0 &= -k^2c^2 + w^2 - b \\ w = w_r + w_i = w_r + 0 = w_r &= +\sqrt{k^2c^2 + b}, \end{aligned}$$

and thus that $w_i = 0 \leq 0$, so the equation is stable and non-dissipative.

Since $v_{ph} = \frac{w_r}{k} = \sqrt{c^2 + \frac{b}{k^2}}$ depends on k, the scheme is dispersive.

3 TR-BDF2

The TR-BDF2 method is an implicit second-order Runge-Kutta method of the following form:

$$\begin{aligned} u_\star &= u_n + \frac{k}{4}\left[f(u_n) + f(u_\star)\right] \\ u_{n+1} &= \frac{1}{3}\left[4u_\star - u_n + kf(u_{n+1})\right] \end{aligned}$$

3.1 L-stability

Applying this method to $f(u) = \lambda u$ yields

$$u_\star - \frac{k}{4}\lambda u_s tar = u_n + \frac{k}{4}\lambda u_n$$
$$u_\star = \frac{1+\frac{k\lambda}{4}}{1-\frac{k\lambda}{4}}u_n = \frac{4+k\lambda}{4-k\lambda}u_n$$

and thus

$$u_{n+1} - \frac{1}{3}k\lambda u_{n+1} = \frac{1}{3}\left[4\frac{4+k\lambda}{4-k\lambda}u_n - \frac{4-k\lambda}{4-k\lambda}u_n\right]$$
$$u_{n+1} = R(\lambda k)u_n,$$

for a stability polynomial given by division below:

$$R(z) = \frac{12+5z}{(4-z)(3-z)} = \frac{12+5z}{z^2-7z+12} = \frac{12+5z}{(12+5z)+(z^2-12z)}.$$

If $Re(\lambda k) = Re(z) < 0$, then adding to $0 \leq -z \leq -12z \leq z^2 - 12z$ yields $12+5z \leq (12+5z) + (z^2-12z)$ and division yields $\frac{12+5z}{(12+5z)+(z^2-12z)} \leq 1$. Thus, the method is A-stable.

By L'Hopital's rule, we also obtain:

$$\lim_{z\to-\infty} R(z) = \lim_{z\to-\infty} \frac{5}{2z-7} = 0.$$

Therefore, the method is L-stable.

3.2 Numerical code

We write a numerical code using TR-BDF2 as the basis for solving the heat equation $u_t = u_{xx}$ in a Method of Lines approach. Using the test case of Homework problem 6.1c, we observe $O(h^2)$ convergence.

[7]:
```
function gen_diags(N, kappa, theta, lambda)
    A = BandedMatrix((-1=>repeat([kappa*theta*lambda],outer=[N-1]),␣
 ↪0=>repeat([1-2*kappa*theta*lambda],outer=[N]) ,␣
 ↪1=>repeat([kappa*theta*lambda],outer=[N-1])), (N,N))
    B = BandedMatrix((-1=>repeat([-(1-theta)*kappa*lambda],outer=[N-1]),␣
 ↪0=>repeat([1+2*(1-theta)*kappa*lambda],outer=[N]) ,␣
 ↪1=>repeat([-(1-theta)*kappa*lambda],outer=[N-1])), (N,N))
    return A, B
end

function forcedheat(u0, T, M, N, theta, kappa)
    s0 = 0.0
    S  = 1.0
    t0 = 0.0
```

```
    h  = S/(N+1)
    k  = T/M
    lambda = k/h^2
    A, B = gen_diags(N, kappa, theta, lambda)
    succ(u_curr) = B\(A*u_curr) # efficient linear solver since matrices are␣
 ↪tridiagonal
    return theta_method(u0, T, M, N, theta, succ, kappa) # BE if theta = 0.0
end

function theta_method(u0, T, M, N, theta, succ, kappa)
    h = 1/(N+1)
    k = T/M
    x = LinRange(0, 1, N+2)
    x_int = x[2:N+1]
    t = LinRange(0, T, M+1)
    lambda = k/h^2

    u = zeros(length(t), length(x))
    u_curr = [u0(x_int[i]) for i in 1:length(x_int)]
    u[1,2:N+1] = u_curr

    A, B = gen_diags(N, kappa, theta, lambda)
    A0, B0 = gen_diags(N, kappa, 0.0, lambda)
    A05, B05 = gen_diags(N, kappa, 0.5, lambda)
    C = BandedMatrix((-1=>repeat([kappa*1.0*lambda],outer=[N-1]),␣
 ↪0=>repeat([-2*kappa*1.0*lambda],outer=[N]) , 1=>repeat([kappa*1.
 ↪0*lambda],outer=[N-1])), (N,N))
    B0 = I- C .* k/3

    for j in range(2, stop=M+1)
        u_star = B05\(A05*u_curr) # trapezoidal
        u_curr = B0\(4*u_star-u_curr) ./ 3
        u_curr = u_star
        u[j,2:N+1] = u_curr
    end
    return (x, t, u)
end

f(x,t) = 0
H = [0.05, 0.02, 0.01, 0.005, 0.002, 0.001]
T = 0.1
kappa = 1.0/20 #1.0/20
J = 5
u3_0(x) = sin(J*pi*x)
u_sol(t,x) = exp(-kappa*t*(J*pi)^2)*sin(5*pi*x)

hb = H[5]
```

```
kb = 2*hb

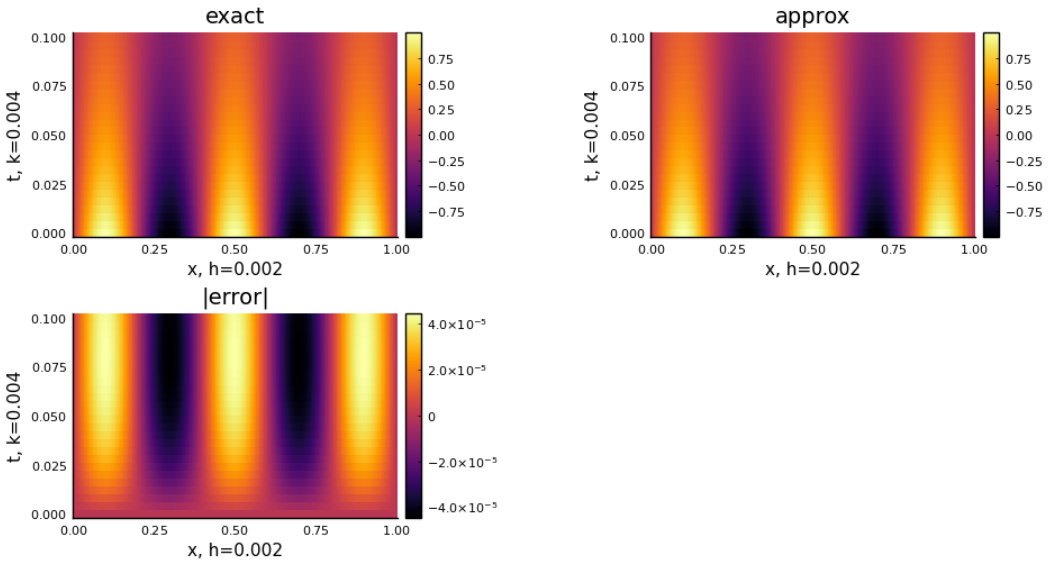
function sol_error(f, u0, kappa, h, k, u_sol, theta)
    S = 1.0
    T = 0.1
    N = Int(round(S/h))
    M = Int(round(T/k))
    x, t, u = forcedheat(u0, T, M, N, theta, kappa)
    u_exact = zeros(length(t), length(x))
    e = zeros(length(t), length(x))
    e_inf = -Inf
    for j in 1:length(t)
        for i in 1:length(x)
            u_exact[j,i] = u_sol(t[j], x[i])
            e[j,i] =  u_exact[j,i] - u[j,i]
            e_inf = max(e_inf, abs(e[j,i]))
        end
    end
    return x, t, u, u_exact, k/(h^2), e, e_inf, e[length(t),:]
end

theta = 0.5
x3, t3, u3, u3_exact, l3, e3, e3_inf, e3T = sol_error(f, u3_0, kappa, hb, kb,␣
 ↪u_sol, theta)
# maximum(abs.(e3T))

function hm(u, title)
    return heatmap(x3, t3, u, title=title, xlabel="x, h=$hb", ylabel="t, k=$kb")
end

plot(hm(u3_exact, "exact"), hm(u3, "approx"), hm(e3, "|error|"), size=(1000,500))
```

[7]:



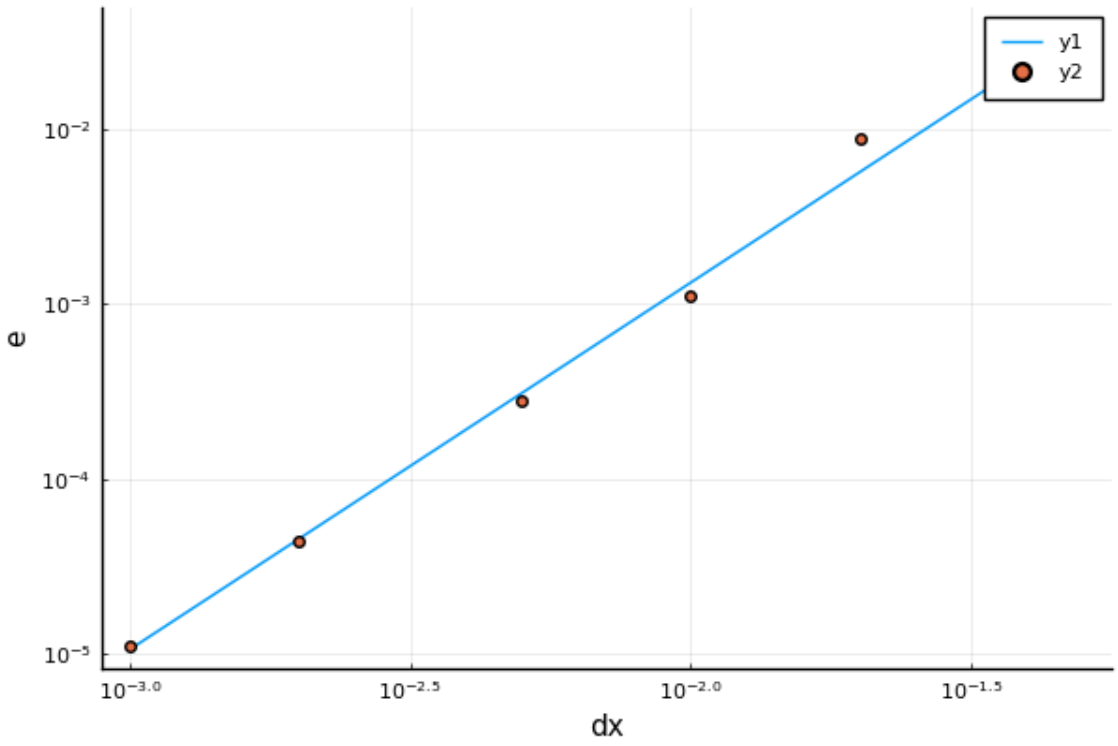
[8]:
```
function heat_error(theta)
    Z = zeros(length(H))
    for c in 1:length(H)
        xc, tc, uc, uc_exact, lc, ec, ec_inf, ec_T = sol_error(f, u3_0, kappa,␣
 ↪H[c], 2*H[c], u_sol, theta)
        e = ec_inf
        Z[c] = maximum(abs.(e))
    end
    return Z
end

function gen_errors()
    theta = 0.5
    return heat_error(theta)
end

Zc = gen_errors()
pc, Cc = regress_convergence(H, Zc)
println((pc, H, Zc))
O(h) = Cc*h^pc
plot(O, minimum(H), maximum(H), xlabel="dx", ylabel="e", xaxis=:log, yaxis=:log)
scatter!(H, Zc)
```

```
(2.0977685615415704, [0.05, 0.02, 0.01, 0.005, 0.002, 0.001],
[0.03216891105208841, 0.008822945448453245, 0.0011293316081060212,
0.00027962466966491295, 4.452284818617924e-5, 1.1114534765155337e-5])
```

[8]:



3.3 Merits

The TR-BDF2 method converges faster than the Backwards Euler method at the cost of additional computation, and it does not produce the same spurrious oscillations as the Crank-Nicolson method does.

4 Finite Volumes

4.1 ODE Solvers

[9]:
```
function forward_euler(f, t0, u0, T, M)
    h = (T-t0)/M
    t = [t0:h:T-h;]
    u = [[0.0, 0.0] for i=1:M]
    u[1] = u0
    u_curr = u0

    for k in range(1, stop=M-1)
        u_curr = u_curr + h * f(t[k], u_curr)
        u[k+1] = u_curr
    end
    return (t,u)
end
```

```
function rk4(f, t0, u0, T, M)
    h = (T-t0)/M
    t = [t0:h:T-h;]
    u = [[0.0, 0.0] for i=1:M]
    u[1] = u0
    u_curr = u0

    for k in range(1, stop=M-1)
        k1 = h*f(t[k], u_curr)
        k2 = h*f(t[k]+h/2, u_curr+k1/2)
        k3 = h*f(t[k]+h/2, u_curr+k2/2)
        k4 = h*f(t[k+1]+h, u_curr+k3)
        u_curr = u_curr + (1/6) * (k1+2*k2+2*k3+k4)
        u[k+1] = u_curr
    end
    return (t,u)
end

function rk2(f, t0, u0, T, M)
    h = (T-t0)/M
    t = [t0:h:T-h;]
    u = [[0.0, 0.0] for i=1:M]
    u[1] = u0
    u_curr = u0

    for k in range(1, stop=M-1)
        k1 = h*f(t[k], u_curr)
        k2 = h*f(t[k]+h/2, u_curr+k1/2)
        u_curr = u_curr + k2
        u[k+1] = u_curr
    end
    return (t,u)
end

function sol_error(exact_sol, approx_method, f, t0, u0, T, M)
    approx = approx_method(f, t0, u0, T, M)
    t, sol = approx
    e_inf = 0 # maximum error observed
    for k in range(1, stop=length(t))
        ek = abs.(exact_sol(t[k])- sol[k])
        ek = sum(ek)
        e_inf = max(e_inf, ek)
    end
    return e_inf
end
```

```
A = [0 -1; 1 0]
f(t,u) = A*u
t0 = 0
u0 = [1, 0]
T = 1
M = 10

function sol(t)
    x(t) = cos(t)
    y(t) = sin(t)
    return [x(t), y(t)]
end

h = [10.0^(o) for o=-2.7:0.01:-2.4]
Z = Dict(
    "forward_euler" => zeros(Float64, length(h)),
    "rk2" => zeros(Float64, length(h)),
    "rk4" => zeros(Float64, length(h)),
)

for k in range(1, stop=length(h))
    M = round(Int, (T-t0)/h[k])
    Z["forward_euler"][k] = sol_error(sol, forward_euler, f, t0, u0, T, M)
    Z["rk2"][k] = sol_error(sol, rk2, f, t0, u0, T, M)
    Z["rk4"][k] = sol_error(sol, rk4, f, t0, u0, T, M)
end

for (k,Zv) in Z
    p, M = regress_convergence(h, Z[k])
    println(k, ": O=(Mh^p) for (p,M)=", (p, M))
end
```

```
forward_euler: O=(Mh^p) for (p,M)=(1.0003092093827588, 0.6913349312723713)
rk4: O=(Mh^p) for (p,M)=(4.003309620864294, 0.011720606097642734)
rk2: O=(Mh^p) for (p,M)=(2.0011070547975933, 0.23136564943398963)
```

[10]:
```
## Upwind Flux

function upwind_flux(u,a)
    Fprev = zeros(length(u))
    if a >= 0
        Fprev = a .* u
    elseif a<0
        Fprev = circshift(a .* u, length(u)-1)
    end
    Fcurr = circshift(Fprev, 1)
    return Fcurr-Fprev
```

```
end

N = 4
u0 = [1,2,3]
a = 2

upwind_flux(u0, a)
```

[10]:
```
3-element Vector{Int64}:
  4
 -2
 -2
```

[11]:
```
## Advection Solver

function advection(u0,a,T,N,M, ode_solver,flux_computer)
# function advection(u0, t0, s0, T, S, M, N, a, A)
    s0, t0, S = 0.0, 0.0, 1.0
    h = S/(N+1)
    k = T/M
    t = Array(LinRange(t0, Float64(T), M+1))
    x = Array(LinRange(s0, S, N+2))
    x_int = x
    u = zeros(length(t), length(x))
    u[1,:] = [u0(x[i]) for i in 1:length(x)]

    u_curr = u[1,:]
#     println(u_curr)
    for j in range(2, stop=M+1)
#         Fprev = flux_computer(u_curr, a)
#         Fnext = circshift(Fprev, 1)
        F = flux_computer(u_curr, a)
        u_curr = u_curr + F .* k/h
        u[j,:] = u_curr

    end
    return (x, t, u)
end

a = 1.0
u4_0(x) = sin(2*pi*x)
u4_e(t,x) = sin(2*pi*(x-t))

function test_fv(exact_sol, t0, u0, T, M, N, ode_solver, flux_computer)
    x, t, u = advection(u0,a,T,N,M, ode_solver, flux_computer)
    e_inf = -Inf # maximum error observed
    e = zeros(length(t), length(x))
```

```
    u_exact = zeros(length(t), length(x))
    for k in range(1, stop=length(t))
        for i in 1:length(x)
            u_exact[k,i] = exact_sol(t[k], x[i])
            ek = u_exact[k,i]- u[k,i]
            e[k,i] = ek
        end
    end
    return t, x, u, u_exact, e
end

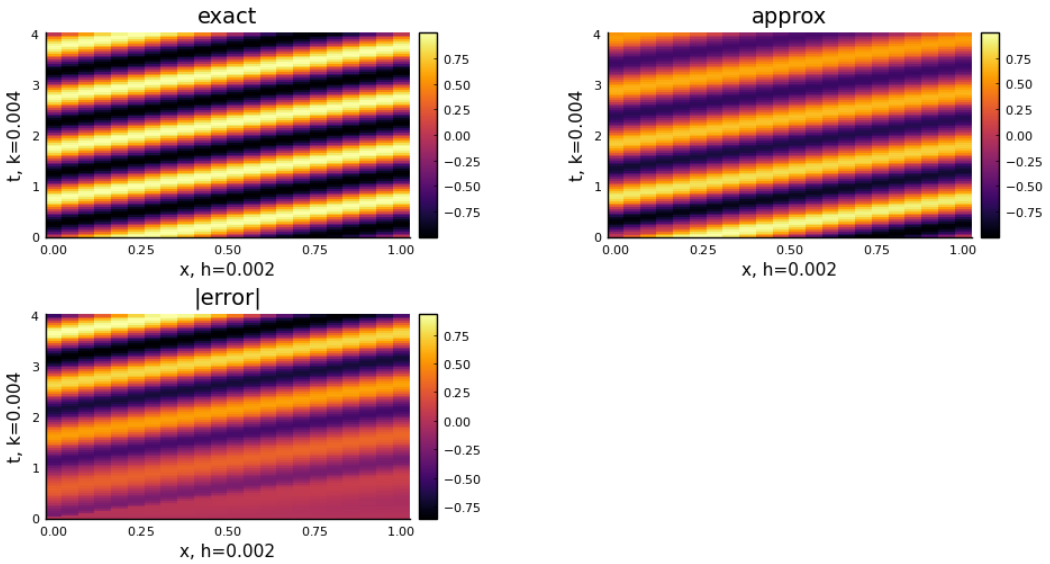

T = 4
M = 100
N = Int(round(M/(T+1)))
t4, x4, u4, u4_exact, e4 = test_fv(u4_e, t0, u4_0, T, M, N, forward_euler,␣
 ↪upwind_flux)

function hm4(u, title)
    return heatmap(x4, t4, u, title=title, xlabel="x, h=$hb", ylabel="t, k=$kb")
end

println(maximum(abs.(e4[M,:])))
plot(hm4(u4_exact, "exact"), hm4(u4, "approx"), hm4(e4, "|error|"),␣
 ↪size=(1000,500))
```

```
0.9234528257884945
```

[11]:

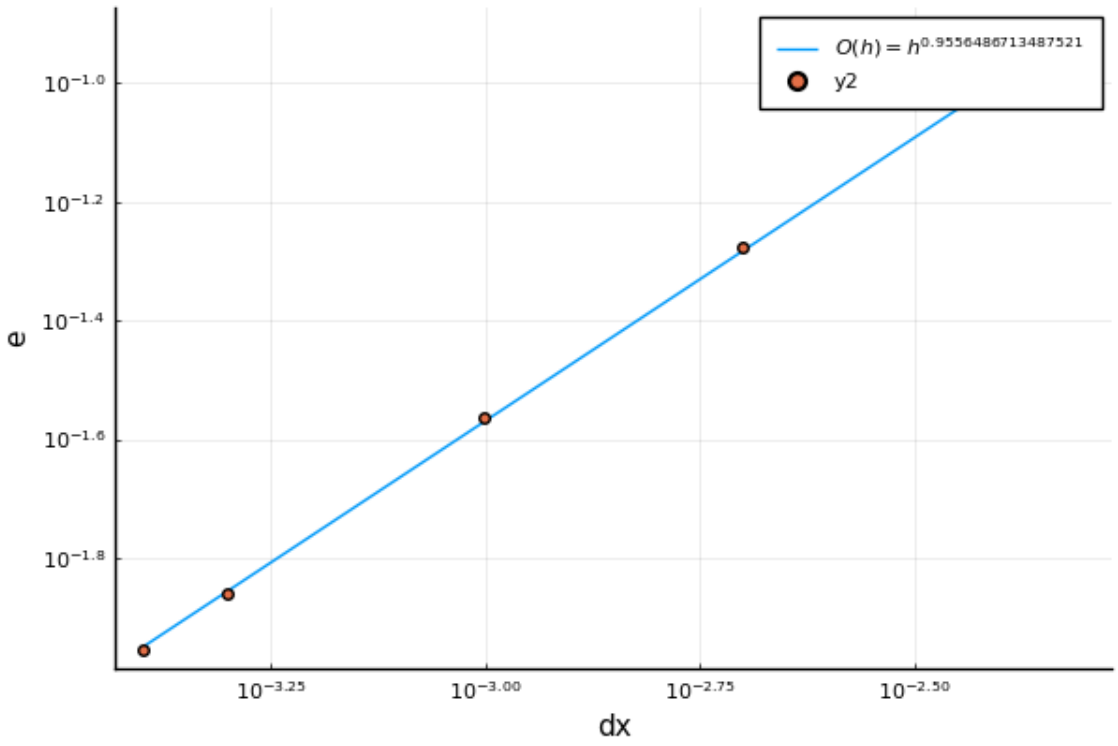


[12]:
```
function get_conv(Ns, ode_solver, flux_computer)
    T = 4
    H, Z = zeros(length(Ns)), zeros(length(Ns))
    for i in 1:length(Ns)
        N = Ns[i]
        M = Int(round(N*(T+1)))
        println((N,M))
        t4, x4, u4, u4_exact, e4 = test_fv(u4_e, t0, u4_0, T, M, N, ode_solver,␣
 ↪flux_computer)
        H[i] = 1.0/(N+1)
        Z[i] = abs.(e4[M])
    end
    return H, Z
end

N4, Z4 = get_conv([200, 500,1000,2000,2500], forward_euler, upwind_flux)
p4, C4 = regress_convergence(N4, Z4)
println((p4, C4))
O4(h) = C4*h^p4
plot(O4, minimum(N4), maximum(N4), xlabel="dx", ylabel="e", xaxis=:log, yaxis=:
 ↪log, label=L"O(h)=h^{%$p4}")
scatter!(N4, Z4, xaxis=:log, yaxis=:log)
```

```
(200, 1000)
(500, 2500)
(1000, 5000)
(2000, 10000)
(2500, 12500)
(0.9556486713487521, 19.83566766553308)
```

[12]:



[13]:
```
function minmod(a,b)
    if a*b <= 0
        return 0
    elseif abs(a)<abs(b)
        return a
    elseif abs(a) > abs(b)
        return b
    end
end

function minmod_flux(u,a)
    Fprev = zeros(length(u))
    if a >= 0
        Fprev = a .* u
    elseif a<0
        Fprev = circshift(a .* u, length(u)-1)
    end
    Fcurr = circshift(Fprev, 1)
    Fnext = circshift(Fcurr, 1)
    return minmod.((Fcurr-Fprev), (Fnext-Fcurr))
end

u0 = [1,2,3]
```

```
a = 1

upwind_flux(u0, a)

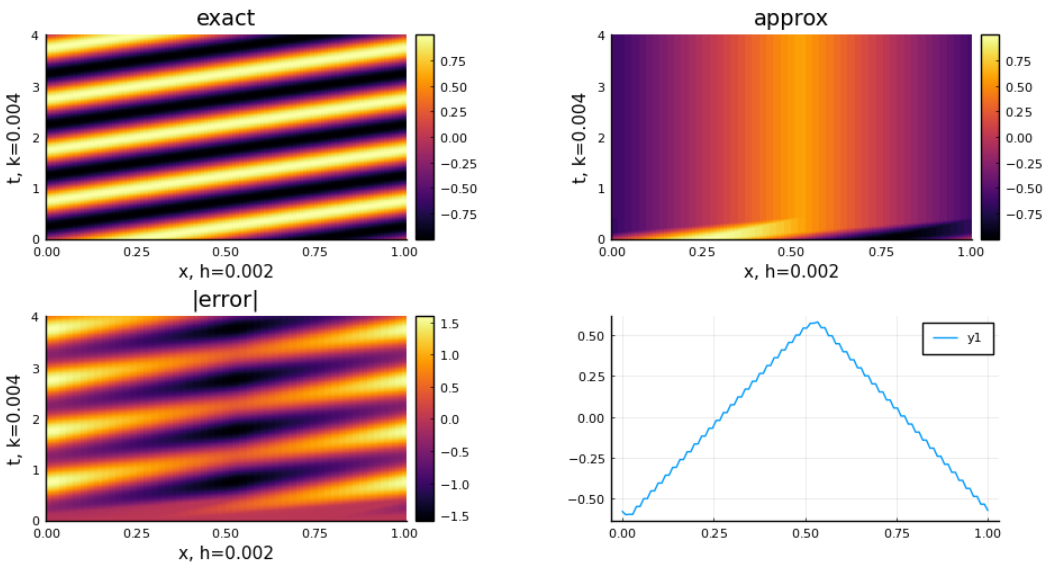
T = 4
M = 500
N = Int(round(M/(T+1)))
t4, x4, u4, u4_exact, e4 = test_fv(u4_e, t0, u4_0, T, M, N, forward_euler,␣
 ↪minmod_flux)

function hm4(u, title)
    return heatmap(x4, t4, u, title=title, xlabel="x, h=$hb", ylabel="t, k=$kb")
end

println(maximum(abs.(e4[M,:])))
plot(hm4(u4_exact, "exact"), hm4(u4, "approx"), hm4(e4, "|error|"), plot(x4,␣
 ↪u4[M,:]), size=(1000,500))
```

```
1.1591846582438414
```

[13]:



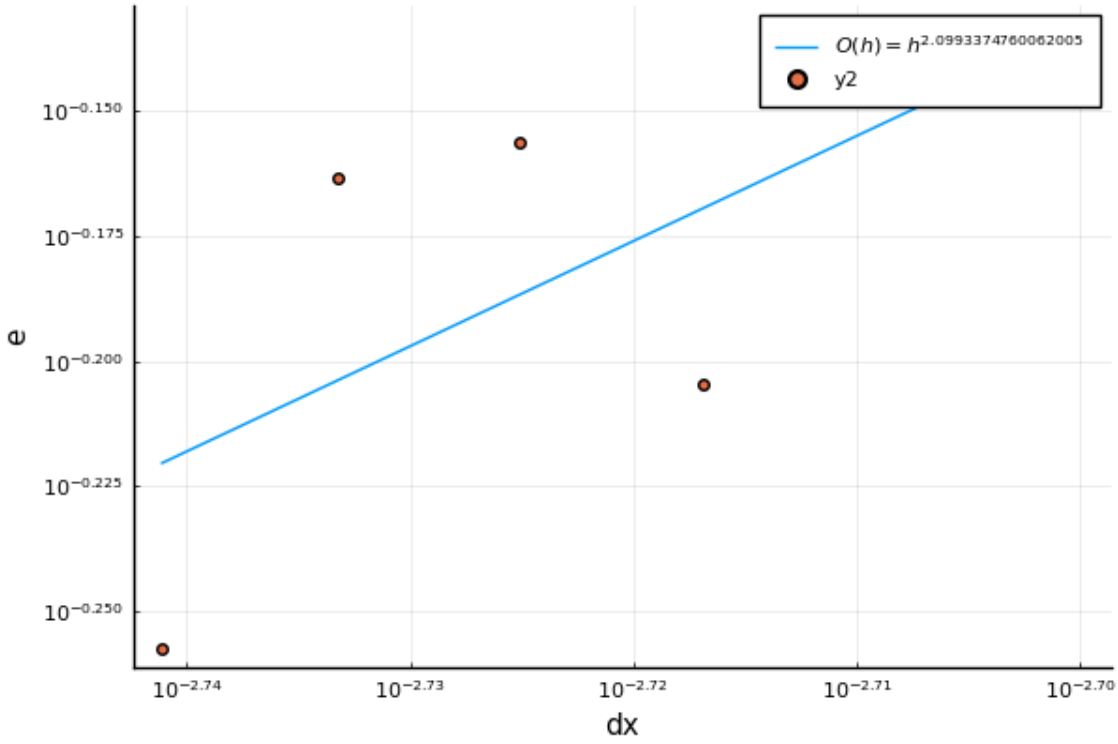
[14]:

```
Nd, Zd = get_conv([500,520,530,540,550], rk4, minmod_flux)
pd, Cd = regress_convergence(Nd, Zd)
println((pd, Cd))
Od(h) = Cd*h^pd
plot(Od, minimum(Nd), maximum(Nd), xlabel="dx", ylabel="e", xaxis=:log, yaxis=:
 ↪log, label=L"O(h)=h^{%$pd}")
scatter!(Nd, Zd, xaxis=:log, yaxis=:log)
```

```
(500, 2500)
```

```
(520, 2600)
(530, 2650)
(540, 2700)
(550, 2750)
(2.0993374760062005, 342096.8207993307)
```

[14]:



We obtained sensible orders of convergence with the upwind flux calculation. With the minmod flux calculation, we seemed to achieve quadratic convergence, but the line does not seem to fit the points well. If there is an error, I suspect it is due to a mistake related to the periodic boundary condition.

Below, we plot the results when the initial condition is a square wave.

[15]:

```
T = 4
N = 100
M = Int(round(N*(T+1)))
a = 1
function ug_0(x)
    if abs(x-1.0/2) < 1.0/4
        return 1
    else
        return 0
    end
end
```

```
ug_e(t,x) = ug_0((x-a*t))
tg, xg, ug_mod, ug_exact, eg = test_fv(ug_e, t0, ug_0, T, M, N, rk4, minmod_flux)
tg, xg, ug_upwind, ug_exact, eg = test_fv(ug_e, t0, ug_0, T, M, N, rk4,␣
 ↪upwind_flux)

function hm4(u, title)
    return heatmap(xg, tg, u, title=title, xlabel="x, h=$hb", ylabel="t, k=$kb")
end

println(maximum(abs.(eg[M,:])))
plot(hm4(ug_exact, "exact"), hm4(ug_upwind, "approx"), hm4(eg, "|error|"),␣
 ↪size=(1000,500))
plot(xg, ug_mod[M,:], size=(1000,500), label="minmod")
plot!(xg, ug_upwind[M,:], label="upwind")
```

```
0.9955873438231295
```

[15]:

